

Data Science Applied to Biomedicine

Project Report

Biomedical Shiny Application with Persistent Storage, Quality Control and Traceability

Student: Oliwia Wójcicka

GitHub repository: <https://github.com/oliwiawojcicka/DataScience-in-biomedicine>

Technology stack: R Shiny, PostgreSQL, Docker, RMarkdown

Date: May 11, 2026

1 Introduction

The aim of this project was to create a biomedical data management application based on hospital admission data. The starting point was a CSV file, but instead of using the file directly during the whole workflow, I migrated the data into a PostgreSQL database. This made the data persistent and allowed the application to work more like a real biomedical information system.

The final application was built in R Shiny. It includes user login, data visualization, patient record management, quality control, a validated form for adding new patients, audit logs, and HTML report generation. The application was implemented with a modular structure, so the main parts of the system are separated into different files.

The project focuses not only on displaying the data, but also on checking whether the data is complete, consistent, and clinically reasonable. This is important because biomedical data often contains missing values, incorrect formats, or unrealistic values that should be detected before analysis.

2 General System Overview

The application consists of three main parts. The first part is the original CSV file with hospital admission records. The second part is the PostgreSQL database, where the data is stored permanently. The third part is the Shiny interface, which allows the user to interact with the database.

The data flow in the project is:

CSV file → R import script → PostgreSQL database → Shiny application

The import is performed using the `init_db.R` script. This script creates the database tables and loads the cleaned CSV data into PostgreSQL. After that, the Shiny application reads and writes data through the database connection. This way, changes made in the application are not temporary and can be tracked.

3 Database Design

The database contains two main tables: `patients` and `audit_logs`. The first table stores the biomedical patient records. The second table stores information about actions performed on the records.

3.1 Patients Table

The `patients` table contains the main data used in the application. The primary key is `patient_id`, which identifies each patient record. The table includes demographic variables, clinical variables, and additional fields used to track when and by whom a record was created or modified.

Table 1: Main columns in the `patients` table.

Column	Description
<code>patient_id</code>	Unique patient identifier and primary key.
<code>date_of_birth</code>	Patient date of birth.
<code>age</code>	Patient age.
<code>sex</code>	Patient sex.
<code>weight</code>	Patient weight in kilograms.
<code>height</code>	Patient height in meters.
<code>blood_type</code>	Patient blood type.
<code>diagnosis_code</code>	Diagnosis code.
<code>dosage_mg</code>	Medication dosage in milligrams.
<code>smoker</code>	Smoking status.
<code>doctor_name</code>	Doctor assigned to the patient.
<code>created_by, created_at</code>	User and time of record creation.
<code>modified_by, modified_at</code>	User and time of record modification.

3.2 Audit Logs Table

The `audit_logs` table is used to keep a history of important actions. This makes it possible to see who imported, added, updated, or deleted a record and when it happened.

Table 2: Main columns in the `audit_logs` table.

Column	Description
<code>log_id</code>	Unique ID of the log entry.
<code>patient_id</code>	Patient affected by the action.
<code>action</code>	Type of action, for example IMPORT, INSERT, UPDATE, or DELETE.
<code>username</code>	User who performed the action.
<code>action_timestamp</code>	Time when the action happened.
<code>old_value</code>	Previous value before update or delete.
<code>new_value</code>	New value after insert or update.

4 Application Structure

The application was divided into separate Shiny modules. I used this approach to make the code more organized and easier to understand. Each important part of the application has its own file in the `R/` folder.

The main modules are:

- `module_dashboard.R` – login logic,
- `module_visualization.R` – basic data visualization,

- `module_records.R` – browsing, filtering, editing, and deleting records,
- `module_quality.R` – data quality checks and indicators,
- `module_form.R` – form for adding new patients,
- `module_reports.R` – HTML report generation,
- `quality_functions.R` – data cleaning, validation, and quality rules,
- `db_connection.R` – database connection and audit log functions.

This modular structure also makes the sidebar navigation clear for the user. The application contains sections for Login, Visualization, Records, Quality, New patient, and Reports.

5 Login and User Access

The application starts with a login panel. I added demo accounts for three user roles: **admin**, **researcher**, and **viewer**. This is a simple user management system, but it shows how different access levels can be introduced in a biomedical application.

Users with permissions can add, update, and delete records. This is important because not every user should be able to modify biomedical data.

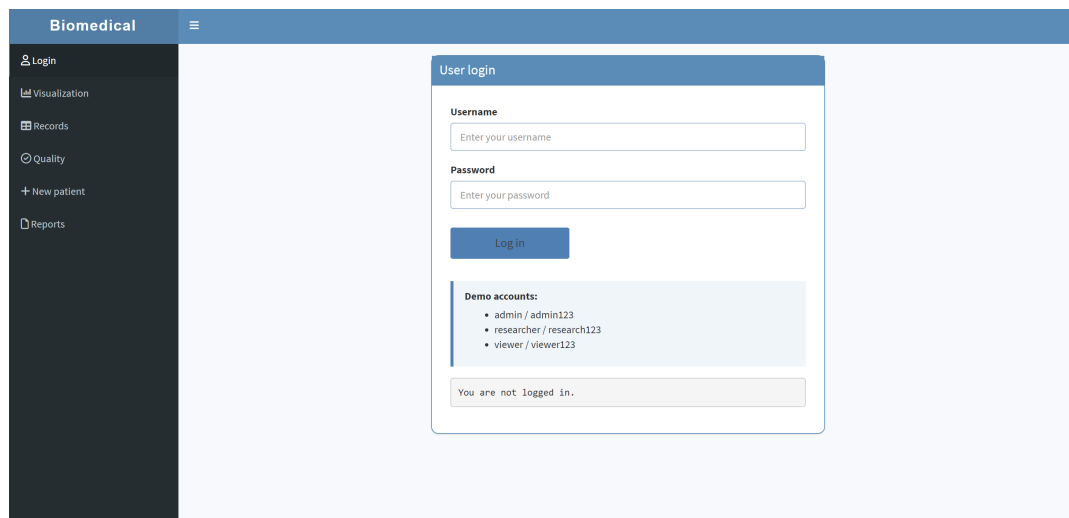


Figure 1: Login screen with demo accounts.

6 Data Visualization

The Visualization section gives a quick overview of the data. I included plots for age distribution, sex distribution, blood type distribution, and dosage by age. These plots help understand the dataset before looking at more detailed quality checks.

The visualizations also make it easier to notice unusual patterns, for example missing categories or extreme dosage values.



Figure 2: Visualization section with age, sex, blood type, and dosage plots.

7 Patient Record Management

The Records section allows the user to work with patient data stored in PostgreSQL. I added a filter panel for the most useful fields: patient ID, sex, blood type, diagnosis code, and doctor name. This is more readable than adding filters to every column in a large table.

The table supports sorting and selecting individual records. After selecting a record, an authorized user can update the dosage and doctor name or delete the record. These changes are not only made in the database, but also recorded in the audit log table.

The figure shows a web interface for managing patient records. It includes a dark blue sidebar with navigation links: Login, Visualization, Records, Quality, + New patient, and Reports.

Record filters: A section with input fields for Patient ID (P-100), Sex (Male), Blood type (All), Diagnosis code (e.g. D123), and Doctor name (e.g. Dr. Smith). A 'Clear filters' button is also present.

Patient records: A table showing patient data. The table has columns: patient_id, date_of_birth, age, sex, weight, height, and blood_type. The table displays two records: P-1008 (2015-03-11, 18, Male, 100, 1.6177811600944, AB+) and P-1009 (2021-07-01, 60, Male, 96, 1.64, O+). A search bar and a 'Show 10 entries' dropdown are also present.

Edit selected record: A section with input fields for 'New dosage [mg]' and 'New doctor name'. A 'Update selected row' button is at the bottom.

Delete selected record: A section with a 'Delete selected row' button. A message states: 'Deleting is available only for admin and researcher users.'

Figure 3: Records section with filters and patient table.

8 Data Quality Control

The Quality section is the main place where the application checks the condition of the database. It works as a quality dashboard. It shows the number of total records, missing values, problematic records, and average completeness.

The section also contains plots for missing values and quality rule violations. This makes it easier to see which variables have the most problems and which validation rules are most often broken.



Figure 4: Quality section with completeness and quality rule violation plots.

8.1 Completeness

Completeness is measured by counting missing values in each variable. This is important because missing clinical data can reduce the usefulness of the dataset. In this project, variables such as blood type, diagnosis code, sex, and smoker status may contain missing values.

8.2 Consistency

Consistency is checked using logical rules. For example, the system checks whether patient ID has a valid format, whether sex has an accepted value, and whether blood type belongs to a valid set of categories.

8.3 Accuracy

Accuracy is checked by detecting unrealistic clinical values. The application flags values such as impossible ages, future dates of birth, unrealistic height or weight, and medication dosages outside the accepted range.

9 Defensive Validation

The New patient form prevents incorrect records from being inserted into the database. When the user enters invalid data, the application displays detailed validation messages and blocks the save operation.

The validation checks include patient ID format, duplicate IDs, future dates of birth, age range, sex category, realistic weight and height, valid blood type, required diagnosis code, and dosage range.

This part of the application is important because it stops obvious errors before they become part of the database.

The screenshot displays the 'Add new patient' form with a dark sidebar on the left containing navigation links: Login, Visualization, Records, Quality, + New patient, and Reports. The main form area is titled 'Add new patient' and contains several input fields: Patient ID (with value 'P-2001'), Date of birth (with value '2026-05-11'), Age, Sex (dropdown), Weight [kg], Height [m], Blood type (dropdown), Diagnosis code, Dosage [mg], and Smoker (dropdown). To the right of the form, a 'Validation details' box is open, showing a list of errors under the heading 'The following validation errors were detected:'. The errors are: Patient ID must have a valid format, for example P-1001 or P1001.; Age must be between 0 and 120.; Sex must be either Male or Female.; Weight must be between 2 and 300 kg.; Height must be between 0.5 and 2.5 meters.; Blood type must be one of: A+, A-, B+, B-, AB+, AB-, O+, O-.; Diagnosis code is required.; Dosage must be greater than 0 and not greater than 5000 mg.

Figure 5: New patient form showing validation errors.

10 Traceability and Audit Logs

The application records important database operations in the `audit_logs` table. During testing, the table stored actions such as data import, new patient insertion, record update, and record deletion.

Each log entry contains the patient ID, action type, username, timestamp, and information about changed values. This makes it possible to check who changed the data and when. In biomedical projects, this type of traceability is important because patient-related records should not be modified without a trace.

	log_id	patient_id	action	username	action_timestamp
1	1049	1177	UPDATE	admin	2026-05-11 14:58:45
2	1048	1112	DELETE	admin	2026-05-11 14:33:33
3	1047	1154	UPDATE	admin	2026-05-11 14:33:25
4	1046	P-20223	INSERT	admin	2026-05-11 14:22:23
5	1045	1159	DELETE	admin	2026-05-11 14:19:42
6	1044	1598	UPDATE	admin	2026-05-11 14:19:35
7	1043	P-9999	INSERT	admin	2026-05-11 14:14:18
8	7	P-11006	IMPORT	csv_import	2026-05-11 14:02:49
9	6	P-1005	IMPORT	csv_import	2026-05-11 14:02:49
10	3	P-1002	IMPORT	csv_import	2026-05-11 14:02:49
11	12	P-1011	IMPORT	csv_import	2026-05-11 14:02:49
12	13	1012	IMPORT	csv_import	2026-05-11 14:02:49
13	14	P-1013	IMPORT	csv_import	2026-05-11 14:02:49
14	15	P-1014	IMPORT	csv_import	2026-05-11 14:02:49
15	11	P-1010	IMPORT	csv_import	2026-05-11 14:02:49
16	10	P-1009	IMPORT	csv_import	2026-05-11 14:02:49
17	5	P-1004	IMPORT	csv_import	2026-05-11 14:02:49
18	19	P-1018	IMPORT	csv_import	2026-05-11 14:02:49
19	20	P-1019	IMPORT	csv_import	2026-05-11 14:02:49
20	18	P-1017	IMPORT	csv_import	2026-05-11 14:02:49

Figure 6: Audit logs showing IMPORT, INSERT, UPDATE, and DELETE actions.

11 HTML Report Generation

The Reports section generates an HTML report using RMarkdown. The report is created from the current data stored in PostgreSQL. It includes a summary of the database, completeness results, missing value analysis, examples of problematic records, and a short description of the quality checks.

The report is generated directly from the Shiny application and can be opened using the link shown in the Reports section.

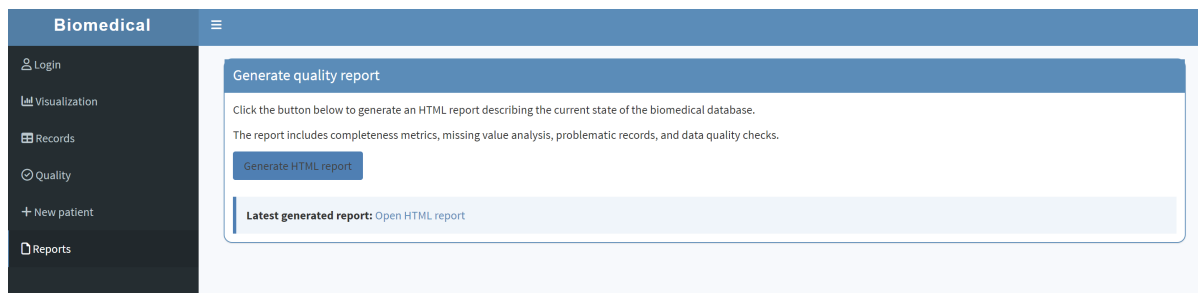


Figure 7: Reports section with generated HTML report link.

12 Discussion

The final application connects a Shiny user interface with a PostgreSQL backend. This means that the data is not only displayed, but also stored persistently and updated through the application.

One important part of the project was that the original CSV data was not perfect. Some records had missing values, inconsistent patient IDs, or unrealistic values. I kept these records in the database and used the Quality section to detect and show these problems. This approach is useful because in real biomedical datasets problematic records should be visible and reviewed, not silently removed.

The application also includes basic user roles and audit logs, which make the system more controlled. Every important modification can be checked later in the database.

13 Conclusion

In this project, I created a biomedical data management application using R Shiny and PostgreSQL. The application imports data from a CSV file, stores it in a database, allows users to view and manage records, checks data quality, validates new patient entries, tracks database changes, and generates an HTML report.

The project shows how a simple CSV dataset can be transformed into a more reliable and traceable biomedical data system.